Sesión 10

Big Data y Formatos de Alto Rendimiento

Carlos César Sánchez Coronel

BASES DE DATOS

Contenido general

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Las 5 V del Big Data

Volumen	Cantidad masiva de datos (terabytes, petabytes). Ej: 500M tweets/día.
Velocidad	Rapidez de generación y procesamiento. Ej: sensores IoT cada segundo.
Variedad	Diferentes formatos: estructurados, semiestructurados, no estructurados.
Veracidad	Calidad y confiabilidad. Ej: datos con ruido o valores erróneos.
Valor	Capacidad de transformar datos en beneficios.

¿Cuándo una solución es Big Data?

- Cuando el volumen supera la capacidad de una sola máquina.
- Cuando se requiere procesamiento en tiempo real (baja latencia).
- Cuando los datos son muy diversos y necesitan integrarse.
- Cuando se necesita escalabilidad horizontal.

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Formatos de archivo

Parquet	Columnar, alta compresión, ideal para análisis en data lakes.
ORC	Columnar, índices internos, usado en Hive/Hadoop.
Avro	Binario orientado a filas, con esquema, excelente para streaming (Kafka).
JSON	Texto semi-estructurado, usado en APIs, pero ineficiente para análisis masivo.
CSV	Texto simple, baja compresión, solo para intercambio básico.

Comparativa de formatos

Formato	Tipo	Compresión	Uso típico
Parquet	Columnar	Alta	Análisis, data lakes
ORC	Columnar	Alta	Hive, Hadoop
Avro	Fila	Media	Streaming, serialización
JSON	Texto	Baja	APIs, documentos
CSV	Texto	Baja	Intercambio simple

Ejemplo: Lectura de Parquet con Spark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("lectura").getOrCreate()
df = spark.read.parquet("s3://mi-bucket/ventas/")
df.filter(df.fecha >= "2025-01-01").groupBy("producto").sum("monto").show()
```

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Data Lake

- Almacena datos en bruto (cualquier formato).
- Bajo costo, flexible (schema-on-read).
- Puede convertirse en un *data swamp* sin gobernanza.
- Ejemplos: S3, HDFS, ADLS.

Data Warehouse

- Datos estructurados y modelados (esquema estrella/copo).
- Alto rendimiento en consultas analíticas (OLAP).
- Costo elevado, menos flexible.
- Ejemplos: Redshift, BigQuery, Snowflake.

Lakehouse

Concepto

Fusión de data lake y data warehouse: almacenamiento tipo lake (bajo costo, formato abierto) con capacidades de gestión y rendimiento de warehouse (transacciones ACID, evolución de esquema).

Tecnologías

- Delta Lake (Databricks)
- Apache Iceberg
- Apache Hudi

Delta Lake

- Capa de almacenamiento ACID sobre Parquet.
- **Time travel**: consultar versiones anteriores.
- **Evolución de esquema**: añadir columnas sin reescribir.
- **Compactación** de archivos pequeños.

Databricks: La plataforma Lakehouse

- Motor Spark optimizado (hasta 50x más rápido).
- Delta Lake como capa de almacenamiento.
- Entorno de notebooks colaborativos.
- Soporte para MLflow (machine learning) y SQL.

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Apache Spark

- Motor unificado: batch, streaming, SQL, ML, Graph.
- Procesamiento en memoria (cuando es posible).
- APIs en Scala, Python, Java, R.
- **DataFrames** y **Spark SQL**.

Apache Flink

- Procesamiento streaming nativo (evento por evento).
- Latencia de milisegundos.
- Excelente manejo de estado y ventanas.

Comparativa Spark vs Flink

Característica	Spark	Flink
Modelo	Micro-batches (o streaming continuo)	Streaming nativo
Latencia	Segundos	Milisegundos
Procesamiento con estado	Sí (checkpoints)	Sí (robusto)
Ecosistema	Muy amplio (SQL, ML, Graph)	En crecimiento

Ejemplo: WordCount con PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col

spark = SparkSession.builder.appName("wordcount").getOrCreate()
df = spark.read.text("s3://bucket/textos/*.txt")
words = df.select(explode(split(col("value"), "\s+")).alias("word"))
counts = words.groupBy("word").count().orderBy(col("count").desc())
counts.show(10)
```

Ejemplo: Streaming con Spark Structured Streaming

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import window, col
from pyspark.sql.types import StructType, StringType, TimestampType

spark = SparkSession.builder.appName("streaming").getOrCreate()

schema = StructType().add("usuario", StringType()).add("timestamp", TimestampType())

df = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "clics") \
    .load()

clics = df.selectExpr("CAST(value AS STRING)") \
          .select(from_json("value", schema).alias("data")) \
          .select("data.*")

counts = clics.groupBy(window("timestamp", "1 minute"), col("usuario")).count()

query = counts.writeStream.outputMode("complete").format("console").start()
query.awaitTermination()
```

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Cloud (Nube)

- Servicios gestionados: AWS EMR, Kinesis, Athena; Google Dataproc, BigQuery; Azure HDInsight, Synapse.
- Escalabilidad elástica, pago por uso.
- Menor mantenimiento, rápida implementación.

On-Premise (Local)

- Infraestructura propia.
- Control total, soberanía de datos.
- Alta inversión inicial (CapEx), mantenimiento complejo.

Híbrida

- Combinación de cloud y on-premise.
- Datos sensibles on-premise, picos de demanda en la nube.
- Ejemplo: data lake en S3, procesamiento crítico en local.

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Redes Sociales (Twitter)

- Cientos de millones de tweets diarios.
- Análisis de tendencias con Spark sobre datos en Parquet.
- Detección de spam y recomendaciones.

Internet de las Cosas (IoT)

- Sensores en plataformas petroleras (ej. Petrobras).
- Ingesta con Kafka, procesamiento en tiempo real con Flink.
- Detección de anomalías y prevención de fallos.

Finanzas (Bancos en Perú)

- BCP, Interbank: millones de transacciones diarias.
- CDC con Debezium, Kafka, Spark Streaming para detección de fraude.
- Almacenamiento en Parquet para reportes regulatorios con Athena.

Retail (Falabella)

- Personalización de ofertas basada en clics y compras.
- Procesamiento en tiempo real con Spark Streaming.
- Modelos de recomendación en Databricks.

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Apache Kafka

- Plataforma de streaming para ingesta y distribución de eventos.
- Cola distribuida, duradera, con particionamiento.
- Se integra con Spark, Flink, etc.

Apache Airflow

- Orquestador de workflows (DAGs en Python).
- Programación y monitoreo de pipelines ETL/ELT.
- Estándar en la industria.

Trino (antes Presto)

- Motor de consultas SQL distribuido.
- Consulta datos directamente en data lakes (S3, HDFS) en formatos como Parquet.
- Ideal para análisis ad-hoc.

Otras herramientas

- **Apache Hive**: Data warehouse sobre Hadoop.
- **Apache HBase**: BD NoSQL columnar para acceso aleatorio.
- **Elasticsearch**: Búsqueda y análisis de texto.

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Ejercicio 1: Elección de formato

Enunciado: Almacenar logs de servidores para análisis posteriores. ¿Qué formato recomendaría?

Solución

Recomendaría **Parquet** por su compresión y eficiencia en consultas analíticas (filtrado por timestamp, nivel de log). Si se requiere ingestión en tiempo real, se puede usar Avro en Kafka y luego convertir a Parquet.

Ejercicio 2: Diseño de un data lake

Enunciado: Estructura de carpetas en S3 para ventas diarias, clientes y productos.

Solución

```
s3://mi-lake/
  raw/
    ventas/
      year=2025/
        month=03/
          day=15/ventas_20250315.csv
          day=16/...
    clientes/clientes_20250315.csv
    productos/productos.csv
  staging/ventas_clean.parquet
  analytics/ventas_diarias.parquet
```

Particionar por fecha facilita consultas y evolución.

Ejercicio 3: WordCount con Spark

Enunciado: Contar palabras más frecuentes en archivos de texto.

Solución (ver código en sección 4)

(El código PySpark ya se mostró en la diapositiva de WordCount).

Ejercicio 4: Procesamiento streaming con Spark

Enunciado: Leer de Kafka eventos de clics y contar por minuto.

Solución (ver código en sección 4)

(El código PySpark de streaming ya se mostró).

Ejercicio 5: Dimensionamiento de clúster

Enunciado: Estimar nodos para procesar 10 TB diarios en 4 horas, con nodos de 64 GB RAM y 8 cores (asumiendo 1 TB/hora cada 10 cores).

Solución

- Capacidad por nodo: 8 cores $\rightarrow$ 0.8 TB/hora.
- Rendimiento necesario: 10 TB / 4 h = 2.5 TB/hora.
- Nodos: 2.5 / 0.8 3.125 $\rightarrow$ **4 nodos**.

(Cálculo aproximado; en la práctica se requieren pruebas de rendimiento).

Contenido de la sección

1. Introducción al Big Data
2. Almacenamiento Optimizado para Big Data
3. Data Lakes, Data Warehouses y Lakehouses
4. Procesamiento Distribuido
5. Arquitecturas Cloud, Híbridas y On-Premise
6. Casos de Uso Reales
7. Herramientas y Plataformas Adicionales
8. Ejercicios Resueltos
9. Glosario

Términos Clave

Big Data	Conjuntos masivos que requieren procesamiento distribuido.
Parquet	Formato columnar de alto rendimiento.
Avro	Formato binario con esquema para streaming.
Data Lake	Repositorio de datos en bruto.
Data Warehouse	Almacén estructurado optimizado para análisis.
Lakehouse	Fusión de data lake y data warehouse.
Delta Lake	Capa ACID sobre Parquet.
Spark	Motor de procesamiento distribuido.
Flink	Motor de streaming de baja latencia.
Kafka	Plataforma de mensajería distribuida.
Airflow	Orquestador de pipelines.
Trino	Motor de consultas SQL sobre data lakes.
CDC	Change Data Capture.
Sharding	Particionamiento horizontal.

¡Gracias!